DIGITAL SYSTEM DESIGN WITH VERILOG USING AI TOOLS

Supplement to Chapter 10

Timing Analysis

Setup, Hold, Critical Path, Fmax — and How to Check Them in Quartus



Case study: the RV32I single-cycle SoC on the DE0 board at 50 MHz
Quartus II · TimeQuest Timing Analyzer · SDC constraints

Agenda

Today's Roadmap

Simulation tells you the logic is right. Timing analysis tells you whether it is still right at the clock speed of the real chip.

Theory 1	Theory 2	Tool	Lab
Sections 01–03	**Sections 04–06**	**Sections 07–08**	**Section 09**
Why timing matters, where delay comes from, setup and hold, slack and Fmax	The critical path of our RV32I, static timing analysis, SDC constraints	TimeQuest step by step, reading a path report, fixing a failing design	Measure Fmax of your SoC, find its critical path, answer the review questions

The one question to carry through the whole lecture

Can every signal leave its flip-flop, pass through the logic and arrive at the next flip-flop before the next clock edge?

01

Why Timing Matters

Why this matters

A design can pass every testbench and still fail on the board. RTL simulation has no delay; real gates and wires do.

Simulation Passed — Why Does the Board Fail?

RTL simulation answers "is the logic correct?". It assumes every signal changes instantly. The chip adds delay to every gate and every wire.

	RTL simulation (Icarus, ModelSim RTL)	Real FPGA
Gate delay	0 ns — outputs change at the clock edge	Each LUT adds a fraction of a ns
Wire delay	0 ns	Routing is often as large as the logic delay
Clock period	Only a number in the testbench	Fixed by the oscillator: 20 ns at 50 MHz
Too-long path	Invisible: result still correct	Wrong value captured → random, hard-to-debug failures

Consequence

Functional verification and timing verification are two different jobs. You need both before the design is done.

Our Case: The RV32I SoC at 1 kHz and at 50 MHz

The same Verilog, the same program. Only the clock changes — and with it, the time each instruction has to finish.

	Version 1 — divided clock	Version 2 — CLOCK_50 directly
CPU clock	1 kHz	50 MHz
Time per instruction	1 ms = 1,000,000 ns	20 ns
Delay for 0.5 s	250 loop iterations	12,500,000 loop iterations
Timing risk	None — any path fits in 1 ms	Every path must finish in 20 ns

The question for this lecture

In a single-cycle CPU the slowest instruction (lw) must fetch, decode, read registers, compute an address, read memory and write back — all inside one 20 ns period. Does it fit?

02

Delay in Real Hardware

Why this matters

Before we can check timing, we need to know where the nanoseconds go: logic, routing and the flip-flops themselves.

Where Delay Comes From

On an FPGA every path is flip-flop → LUTs and routing → flip-flop. Each piece adds delay.

Clock-to-Q (tco)

After the clock edge, the flip-flop output needs time to change.

Size small, fixed per device

Logic delay

Each LUT the signal passes through adds delay. Deep logic = many LUT levels.

Grows with adders, wide MUXes, decoders

Routing delay

Wires between LUTs are programmable switches — they are slow.

Often as large as the logic delay

Setup / Hold (tsu, th)

The receiving flip-flop needs the input stable around its clock edge.

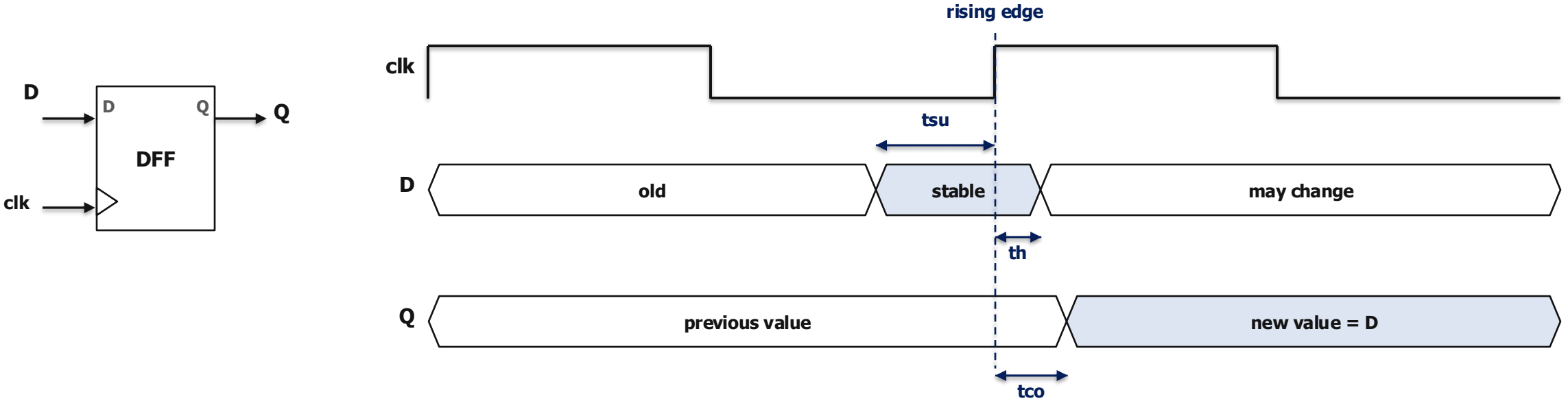
Size small, fixed per device

tco → **+ logic** → **+ routing** → **+ tsu** → **≤ clock period**

This single inequality is the whole setup check. The rest of the lecture explains each term and how the tool measures it.

Flip-Flop Timing: tsu, th and tco

A flip-flop samples D at the rising edge. D must be stable from tsu before the edge until th after it. Q changes tco after the edge.



If D changes inside the tsu–th window

The flip-flop may capture the old value, the new value — or become metastable for a while. Timing analysis exists to guarantee this never happens.

Two Delays per Path: Longest and Shortest

Every combinational path has a slowest case and a fastest case. The two timing checks use different ones.

Propagation delay — tpd (max)

- The LATEST time the output can settle after the input changes.
- Set by the slowest route through the logic (e.g. a full 32-bit carry chain).
- Used for the SETUP check: data must arrive before the next edge.

Contamination delay — tcd (min)

- The EARLIEST time the output can start to change.
- Set by the fastest route (e.g. a single LUT between two registers).
- Used for the HOLD check: data must not arrive too soon.

Rule of thumb

Setup problems come from long paths and fast clocks. Hold problems come from very short paths and clock skew — not from the clock frequency.

03

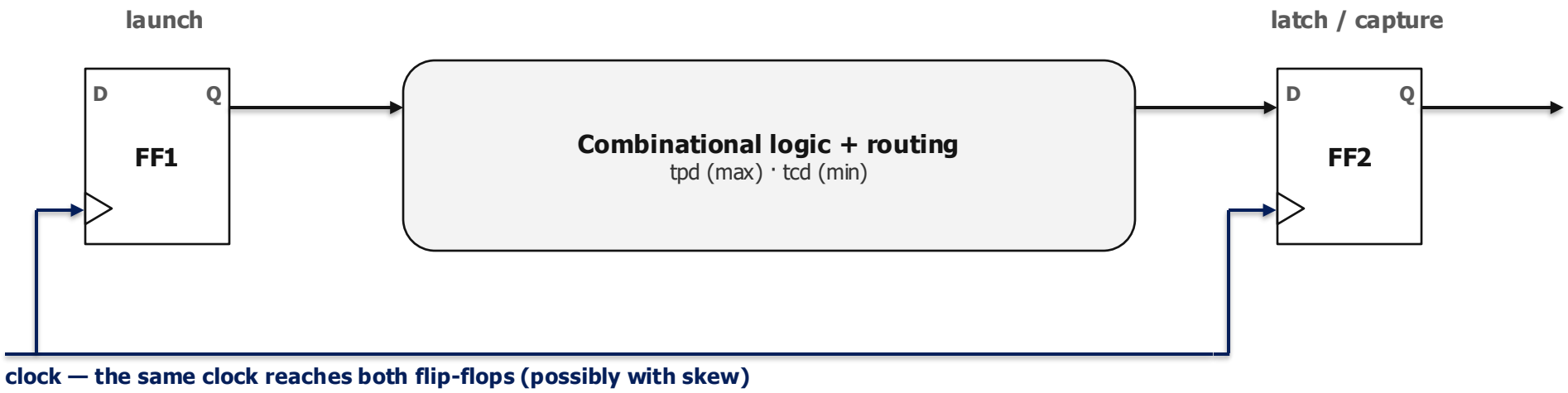
Setup and Hold

Why this matters

Two inequalities decide whether a synchronous design works. Everything the timing analyzer reports is built on them.

The Register-to-Register Path

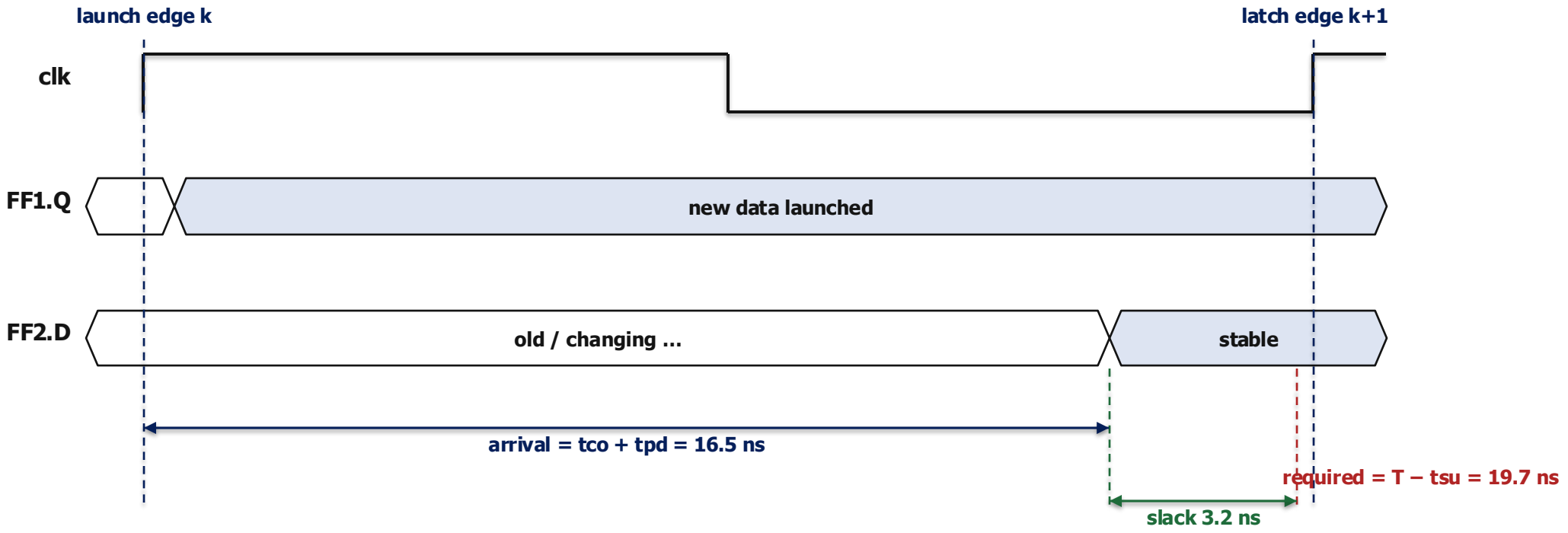
Launch: FF1 sends new data at clock edge k. Latch: FF2 must capture it correctly at edge k+1, one period later.



Check	Question	Uses
Setup	Does the NEW data arrive at FF2 early enough for edge k+1?	tco + tpd (max) + tsu, clock period
Hold	Does the NEW data arrive late enough not to disturb FF2 at edge k?	tco + tcd (min), th — not the period

The Setup Check

Data launched at edge k must be stable at FF2 at least tsu before edge k+1. The time left over is the setup slack.




```
Setup slack = required − arrival = (T − tsu) − (tco + tpd)      must be ≥ 0
```

Worked Example: Slack and Fmax

Illustrative numbers for one long path (they are not measured values). Change only the clock and watch the slack.

Term	Value
tco (FF1)	0.5 ns
Logic delay (max)	12.0 ns
Routing delay (max)	4.0 ns
tsu (FF2)	0.3 ns
Minimum period Tmin	**16.8 ns**

```
Tmin = tco + tpd + tsu = 16.8 ns
Fmax = 1 / Tmin        = 59.5 MHz
```

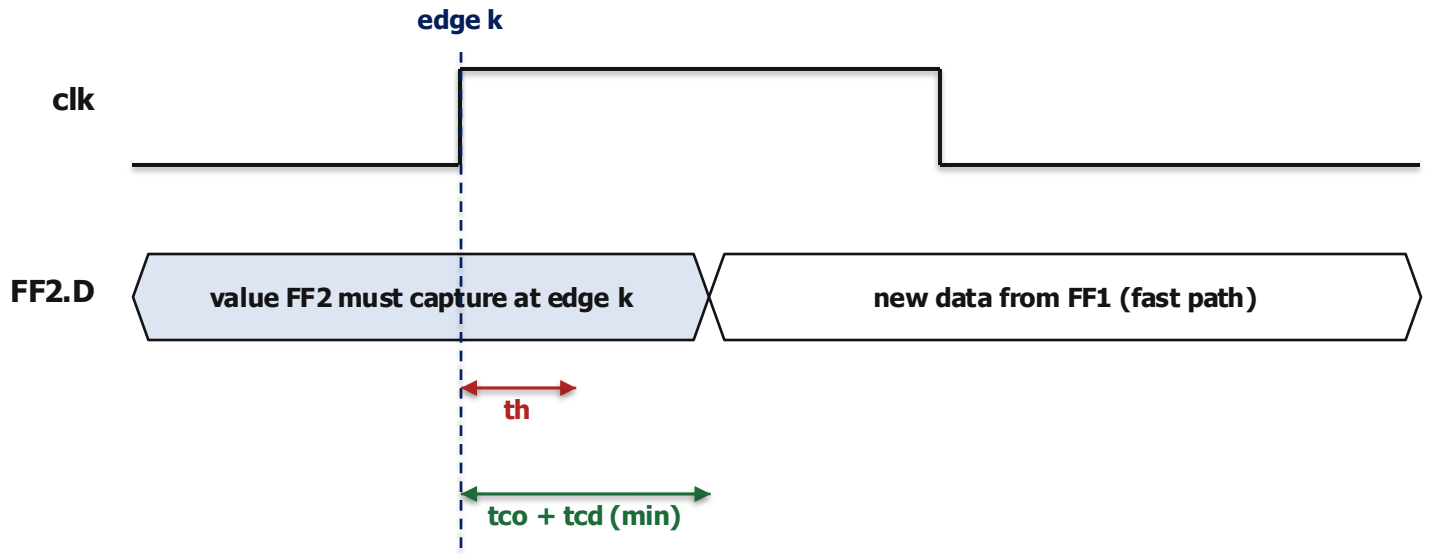
Clock	Period T	Setup slack	Result
25 MHz	40 ns	+23.2 ns	**Pass**
50 MHz	20 ns	+3.2 ns	**Pass**
59.5 MHz	16.8 ns	0 ns	**Limit**
100 MHz	10 ns	−6.8 ns	**Fail**

Slack = T − Tmin

Positive slack is margin. Negative slack means some flip-flop will capture wrong data at that clock.

The Hold Check

The new data from edge k must not reach FF2 so fast that it overwrites what FF2 is capturing at the same edge k.




```
Hold slack =
  (tco + tcd) − th
  must be ≥ 0
```

The clock period does not appear in the hold check

Slowing the clock fixes a setup failure but can never fix a hold failure. Hold is fixed by adding delay to the short path — the Fitter does this automatically on FPGAs.

With one global clock and no skew, FPGA flip-flops normally meet hold by construction. Hold failures appear with skew or badly built clocks.

Clock Skew and Clock Uncertainty

Skew: the clock reaches FF1 and FF2 at slightly different times. Uncertainty: jitter and other effects the tool must allow for.

Effect	On setup	On hold
Capture clock arrives later (positive skew)	Helps — more time for data	Hurts — FF2 samples later, fast data can overrun it
Capture clock arrives earlier (negative skew)	Hurts — less time for data	Helps
Clock uncertainty (jitter)	Hurts — subtracted from the period	Hurts — added to the hold requirement

```
Setup:  tco + tpd + tsu  ≤  T + tskew − tuncertainty
Hold:   tco + tcd        ≥  th + tskew + tuncertainty
```

Why FPGAs use global clock networks

Dedicated clock trees keep skew tiny. A clock made from ordinary logic has large skew — one reason to avoid it (Section 08).

04

The Critical Path of RV32I

Why this matters

The critical path is the slowest register-to-register path. It alone sets Fmax — for a single-cycle CPU, that is the load instruction.

Single Cycle: One Clock for the Slowest Instruction

Every instruction gets the same clock period, so the period must fit the instruction that uses the most hardware in series.

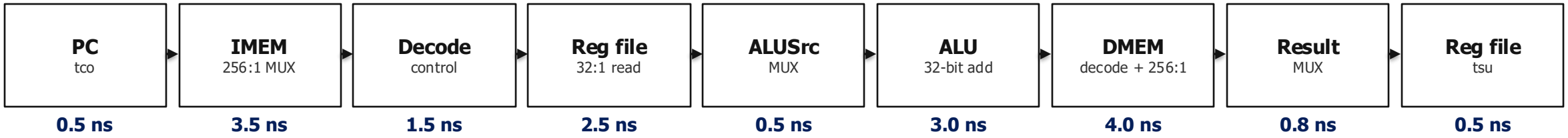
Instruction	IMEM	Decode	Reg read	ALU	DMEM read	Result MUX	Reg write
R-type (add)	•	•	•	•		•	•
lw	•	•	•	•	•	•	•
sw	•	•	•	•	write		
beq / bne	•	•	•	•			→ PC
jal	•	•				•	•

lw uses every block, one after another

PC → IMEM → decode → register file → ALU (address) → address decoder → DMEM → result MUX → register file. That chain is the critical path candidate.

The lw Path, Block by Block

Rough estimate including routing — illustrative only. In the lab you replace these numbers with what TimeQuest reports.





Total ≈ 16.8 ns → slack ≈ +3.2 ns at 50 MHz (estimate)

Observation	Why it matters
The two memories are the biggest pieces	Built from LUTs, a 256-word read is a large multiplexer tree (dark bars)
Routing is inside every number	On an FPGA the wires between blocks can cost as much as the blocks
A small margin is fragile	Adding a peripheral or a wider MUX can turn +3 ns into a failure

Why Our Memories Are on the Critical Path

IMEM and DMEM use a combinational read so that fetch, execute and load all happen in one cycle. That choice has a timing cost.

Memory from logic (our design)

- **Read** combinational: address in → data out in the same cycle
- **Built from** LUTs and registers — a 256:1 multiplexer per bit
- **Timing** slow and grows with memory size
- **Fits** single-cycle CPU, small memories

Memory in M9K blocks

- **Read** synchronous: data appears after the next clock edge
- **Built from** dedicated RAM blocks on the Cyclone III
- **Timing** fast, fixed delay, no LUT tree
- **Fits** multi-cycle or pipelined CPU (fetch stage)

Design trade-off

Faster memory changes the CPU: the instruction arrives one cycle later, so the core needs an extra fetch cycle or a pipeline. Timing and architecture are linked.

05

Static Timing Analysis

Why this matters

STA checks every path in the design without simulation vectors. It is how Quartus proves — or disproves — that 50 MHz works.

Static Timing Analysis vs Timing Simulation

STA adds up delays along every path in the netlist. It does not need a testbench and it cannot miss a path.

	Timing (gate-level) simulation	Static timing analysis (TimeQuest)
Input	Netlist + delays + test vectors	Netlist + delays + constraints (SDC)
Coverage	Only the paths your vectors exercise	Every path, every corner
Speed	Slow — simulates every cycle	Fast — seconds to minutes
Answer	"Did these cycles work?"	"Will every cycle work at this clock?"
Checks logic?	Yes	No — use RTL simulation for that

The complete sign-off

RTL simulation proves the function. STA proves the timing. Together they replace running a slow gate-level simulation for every test.

Timing Models: Corners

Delay changes with voltage and temperature. Quartus analyzes several operating corners and the design must pass all of them.

Corner (Cyclone III)	Chip behaves	Most important for
Slow 1200mV 85C Model	Slowest: hot, lowest voltage	**Setup and Fmax — read this first**
Slow 1200mV 0C Model	Slow, but cold	Setup at low temperature
Fast 1200mV 0C Model	Fastest: cold, highest voltage	**Hold — shortest delays**

Which number is Fmax?

Report the Fmax from the Slow 85C model. That is the guaranteed speed over the full operating range, not the best case.

Corner names are shown under Compilation Report → TimeQuest Timing Analyzer in Quartus II 13.1.

06

Constraints (SDC)

Why this matters

Timing analysis compares delays against requirements. The requirements come from you, in an SDC file. Without it, the numbers mean nothing.

Why the Tool Needs Constraints

Quartus can measure delays by itself, but it cannot know how fast CLOCK_50 is, or which inputs are asynchronous. You must tell it.

Without an SDC file

- Quartus prints Critical Warnings about missing constraints
- Clocks are analyzed against a default requirement, not 20 ns
- Slack and pass/fail do not describe your board
- The Fitter does not know which paths to optimize

With an SDC file

- Every clock has the real period (20 ns)
- Slack is measured against the real requirement
- The Fitter works harder on the paths that fail
- Reports tell you exactly which path is too slow

Rule

No SDC, no timing sign-off. Treat a missing-constraint warning as an error.

SDC Essentials

SDC (Synopsys Design Constraints) is a Tcl-based format shared by most FPGA and ASIC tools. A small set of commands covers a class project.

```
# 1. Define the board clock: 50 MHz = 20 ns
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]

# 2. Clocks made by a PLL (if you use one)
derive_pll_clocks

# 3. Let the tool add jitter / uncertainty
derive_clock_uncertainty

# 4. Asynchronous inputs: no timing relationship
set_false_path -from [get_ports {BUTTON[*] SW[*]}]
```

Command	Meaning
create_clock	A clock enters on this pin with this period
derive_pll_clocks	Create the PLL output clocks automatically
derive_clock_uncertainty	Add the device's jitter allowance
set_false_path	Do not time this path (only when it is truly asynchronous)

Use set_false_path with care

It hides a path from the analyzer. Apply it only to inputs that go through a synchronizer, never to hide a real failure.

Our Constraint File: soc_top.sdc

The LED-only SoC has one clock and no inputs besides the clock, so the file is three lines.

```
# soc_top.sdc -- timing constraint for Quartus (add to the project)
create_clock -name CLOCK_50 -period 20.000 [get_ports CLOCK_50]
derive_clock_uncertainty
```

Item	Why
`-period 20.000`	The CPU runs directly on CLOCK_50 = 50 MHz, so every path must fit in 20 ns
`get_ports CLOCK_50`	The name must match the top-level port name exactly
`No false paths`	LEDG is an output only; the reset comes from an internal power-on counter
`Lab experiment`	Change 20.000 to 10.000 and recompile: the same design now fails — slack goes negative

07

Timing Analysis in Quartus

Why this matters

The analysis runs automatically at the end of every full compilation. Your job is to give it the SDC, then read the right reports.

The Workflow

Quartus II 13.1 calls the tool TimeQuest Timing Analyzer; Quartus Prime calls it Timing Analyzer. The steps are the same.



Step	Where in Quartus
1 Add SDC	Assignments → Settings → TimeQuest Timing Analyzer → add soc_top.sdc
2 Full compile	Processing → Start Compilation (Ctrl+L) — Analysis & Synthesis alone is not enough
3 Check messages	Messages window: no warning about missing clock constraints
4 Read summaries	Compilation Report → TimeQuest Timing Analyzer → Slow 1200mV 85C Model
5 Worst path	Tools → TimeQuest Timing Analyzer → Report Top Failing Paths / Report Timing
6 Fix & repeat	Change the design or the clock, recompile, read the reports again

Steps 1–3: Constrain, Compile, Check

Timing is analyzed on the placed and routed design, so it needs a full compilation — the Fitter decides the routing delays.

1 Add the SDC file

Copy soc_top.sdc into the project folder.

Menu Assignments → Settings → TimeQuest Timing Analyzer

Then … → select file → Add → OK

2 Full compilation

Processing → Start Compilation.

Runs Analysis & Synthesis → Fitter → Assembler → TimeQuest

Note TimeQuest needs the Fitter's routing

3 Check the messages

Filter the Messages window for Critical Warnings.

Bad sign no clocks / no SDC found

Bad sign Timing requirements not met

Before reading any number

Confirm the SDC was read and CLOCK_50 appears in the Clocks report with period 20.000. Otherwise every later step is meaningless.

Step 4: Read the Summaries

Compilation Report → TimeQuest Timing Analyzer. Open the Slow 1200mV 85C Model first — it is the worst case for setup.

Report	What it shows	The design passes when
Clocks	Every clock the tool found and its period	CLOCK_50, period 20.000 ns
Fmax Summary	Highest frequency the worst path allows	Fmax ≥ 50 MHz
Setup Summary	Worst setup slack per clock	Slack ≥ 0 (positive or zero)
Hold Summary	Worst hold slack per clock	Slack ≥ 0 — check the Fast 0C model too
Unconstrained Paths	Paths no constraint covers	Nothing important left unconstrained

Reading slack

Example: Fmax 62 MHz and setup slack +3.9 ns → the worst path is 3.9 ns faster than needed: 50 MHz works. Fmax 41 MHz → slack about −4.4 ns: it does not.

Step 5: Find the Worst Path

Summaries say whether you pass. Path reports say why — which flip-flop, which logic, how many nanoseconds.

Open Tools → TimeQuest Timing Analyzer

Tasks pane double-click Report Top Failing Paths (only if slack < 0)

Or Custom Reports → Report Timing: clock CLOCK_50, Setup, 10 paths

Click a path see the Data Arrival and Data Required panels

Right-click Locate Path → Technology Map Viewer to see the logic

What to expect for our RV32I

- From: u_core|pc[…] (the PC register)
- To: u_core|u_rf|rf[…] (a register-file entry)
- Through: IMEM, decode, register file, ALU, DMEM, result MUX
- In other words: the lw path from Section 04

If the worst path surprises you

That is useful information: it points to logic you did not expect to be slow — often a wide MUX or a long decoder.

Reading a Path Report

Illustrative report in the TimeQuest layout. The slack is simply the Data Required time minus the Data Arrival time.

```
Slack             :  3.214
From Node         :  soc:u_soc|rv32i_core:u_core|pc[3]
To Node           :  soc:u_soc|...|regfile:u_rf|rf[9][31]
Launch Clock      :  CLOCK_50  (rise)
Latch Clock       :  CLOCK_50  (rise)
Relationship      :  20.000
Clock Skew        :  -0.040
Data Delay        :  16.690

Data Arrival Time   :  19.642
Data Required Time  :  22.856
```

Field	Meaning
Relationship	Time between launch and latch edges = the period
Clock Skew	Latch clock minus launch clock arrival
Data Delay	tco + logic + routing along this path
Arrival / Required	Absolute times incl. clock-network delay
Slack	Required − Arrival: ≥ 0 passes

Numbers are illustrative. Arrival and Required include the clock-network delay, so they are larger than 20 ns — only their difference matters.

08

Fixing Timing

Why this matters

When slack is negative there are only a few real options: less work per clock, more time per clock, or faster logic.

Options When Setup Slack Is Negative

Ordered from easiest to most invasive. For a class project, the first row is usually the right answer.

Option	What it does	Cost
Lower the clock (PLL)	More time per cycle, e.g. 25 MHz = 40 ns	Program runs slower; adjust software delays
Faster memories (M9K)	Removes the LUT MUX trees from the path	Synchronous read → CPU needs an extra cycle
Multi-cycle CPU	Each instruction takes several short cycles	More control logic (FSM)
Pipelined CPU	Registers split the path into stages	Hazards: forwarding, stalls, flushes
Fitter settings	Optimization mode: speed; more fitter effort	Longer compile; small gains only

What does not work

Adding set_false_path or a larger period in the SDC only hides the problem. The board still runs at 50 MHz — the constraint must describe reality.

Do Not Build Clocks from Logic

A counter output used as a clock is a "derived" or "ripple" clock: it has large skew, can glitch, and makes timing analysis harder.

Logic clock

```
// Avoid: logic-generated clock
always @(posedge CLOCK_50) div <= div + 1;
assign slow_clk = div[15];
always @(posedge slow_clk) ...   // new clock domain
```

Clock enable

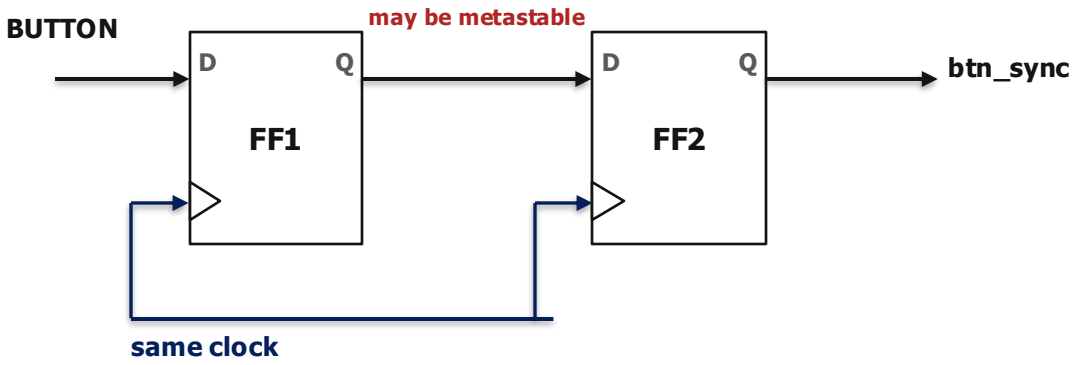
```
// Prefer: one clock + clock enable
wire en = (div == 0);
always @(posedge CLOCK_50)
  if (en) state <= next_state;   // same clock
```

	Logic clock	Clock enable	PLL (ALTPLL)
Skew	Large — ordinary routing	None — global clock	None — global clock
Glitches	Possible	No	No
Timing analysis	Needs extra constraints	Automatic	derive_pll_clocks
Use for	Avoid	Slow logic in a fast domain	A different real frequency

Our 50 MHz SoC uses CLOCK_50 directly and does the 0.5 s timing in software — no derived clock at all.

Asynchronous Inputs and Metastability

Buttons and switches change at any time — timing analysis cannot guarantee setup or hold for them. A synchronizer contains the risk.



Metastable FF1 may sit between 0 and 1 for a while

FF2 gives it a whole clock period to settle

Result the rest of the design sees a clean, synchronous signal

In SDC set_false_path from the input pin to FF1 only

In the Chapter 10 SoC

The full GPIO version synchronizes SW and KEY with two flip-flops (gpio.v). The LED-only version has no inputs, so it needs no synchronizer and no false path.

09

Lab and Review

Why this matters

Measure the real timing of the SoC you built, find its critical path, and explain the result with the equations from this lecture.

Lab: Measure Your SoC

Use the LED-only RV32I SoC running on CLOCK_50. Record every number — you will need them for the review questions.

#	Task	Record
1	Add soc_top.sdc and run a full compilation	Any critical warnings?
2	Clocks report: is CLOCK_50 at 20.000 ns?	Clock name and period
3	Fmax and setup slack for the three corners	Worksheet on the next slide
4	Report Timing: worst setup path	From node, To node, data delay
5	Is the worst path the lw path? Which blocks does it cross?	Sketch it on the datapath
6	Program the board: does the LED count every 0.5 s?	Yes / No

Deliverable

The filled worksheet, a screenshot of the Fmax Summary and of the worst path, and a two-sentence explanation of the slack.

Lab Worksheet

Fill in the values from Compilation Report → TimeQuest Timing Analyzer.

Timing model	Fmax (MHz)	Setup slack @ 20 ns	Hold slack	Pass?
Slow 1200mV 85C				
Slow 1200mV 0C				
Fast 1200mV 0C				

Worst setup path	
From node	
To node	
Data delay (ns)	

Three Experiments

Change one thing at a time, recompile and compare with your worksheet. Predict the result before you run it.

A Tighter clock

Change the SDC period from 20.000 to 10.000 (100 MHz).

Predict setup slack at 10 ns

Check Report Top Failing Paths

Note the hardware did not change

B No constraints

Remove soc_top.sdc from the project and recompile.

Look for Critical Warnings in Messages

Ask what requirement was analyzed?

Restore the SDC afterwards

C Bigger memory

Change DMEM from 256 to 1024 words and recompile.

Predict does Fmax go up or down?

Explain using the 256:1 → 1024:1 MUX tree

Compare the worst path before and after

Why predict first

A prediction that is wrong is the most useful result of the lab — it shows which part of the timing model you have not understood yet.

Review Questions

Answer without looking back. Use the equations, not the tool, for questions 1–4.

1. A path has tco = 0.4 ns, tpd = 15 ns, tsu = 0.2 ns. What is Fmax? What is the slack at 50 MHz?
2. Why does slowing the clock fix a setup violation but never a hold violation?
3. Which RV32I instruction sets the clock period of a single-cycle CPU, and why?
4. The capture clock arrives 0.5 ns later than the launch clock. Does this help setup or hold?
5. Why should Fmax be taken from the Slow 85C model?
6. What happens to the timing report if the project has no SDC file?
7. Why is a clock enable better than a clock made from a counter bit?
8. Moving IMEM to M9K makes it faster. What does it force you to change in the CPU?

Final Takeaways

Six ideas to keep from this lecture.

Two checks

Setup: data must arrive before the next edge. Hold: data must not arrive too soon.

One number

Slack = required − arrival. Positive passes, negative fails.

Fmax

1 / (tco + tpd + tsu) of the worst path — read it from the Slow 85C model.

Critical path

In a single-cycle RV32I it is lw: PC → IMEM → registers → ALU → DMEM → registers.

Constraints

No SDC, no sign-off. create_clock with the real period is the minimum.

Clocks

Use the board clock, clock enables or a PLL — never a clock made from logic.